

SCOUTAI

Instant Competitive Intelligence, On Demand

"Type what you want to know about your competitors. ScoutAI scrapes the web and hands you a boardroom-ready intelligence brief in 60 seconds."

Hackathon Project Brief | Team of 8 | 24 Hours

Confidential — Internal Use Only

PAGE 1 | ONE-PAGE PROJECT OVERVIEW

ScoutAI — The Intelligent Competitive Brief Engine

The problem no tool has solved: A business user with a plain-English question about their competitors cannot get a structured, narrative answer in under a minute — without a developer, a dashboard setup, or a six-figure contract.

The Gap We Are Filling

Today, competitive intelligence sits at two broken extremes:

- **Too big:** Enterprise CI platforms (Crayon, Klue, Contify) — powerful but expensive, slow to set up, and built for dedicated CI teams. Only 11.4% of companies have those.
- **Too raw:** Raw scraping tools (Firecrawl, Scrapy, Octoparse) — return raw HTML or JSON. Developers love them. Business users cannot use them.

The missing middle is a tool that accepts a plain-English objective, scrapes the right sources automatically, and returns a structured, readable competitive brief. That is ScoutAI.

What ScoutAI Does

Dimension	What Exists Today	What ScoutAI Does
User Input	A URL or keyword	A plain-English business objective
Discovery	User must supply URLs	AI auto-discovers competitors
Processing	Raw scraped data dumped	Firecrawl → OpenAI synthesis pipeline
Output	JSON / tables / HTML	Structured narrative brief + visual summary
Gap Insight	None	AI flags what NO competitor is saying — your opening
Audience	Developers only	Any business user, zero setup
Time to Value	Hours to days	Under 60 seconds

The Tech Stack

Tool	Role	What It Does in ScoutAI
Bolt	Frontend UI	Input interface, results card, brief viewer — zero config frontend

Firecrawl API	Web Scraper	Scrapes competitor pages to clean markdown — handles JS, anti-bot, multi-URL
OpenAI GPT-4o	AI Brain	Synthesises scraped content into structured brief + gap analysis
Supabase	Backend / DB	Stores user briefs, enables history, handles auth

The 60-Second Demo Moment

A judge types: 'I am launching a project management SaaS. Who are the top 3 competitors and what are they NOT telling their customers?' ScoutAI scrapes 3–5 sites via Firecrawl, runs GPT-4o synthesis, and returns: a competitive positioning summary, a messaging gap map, and the exact white space the judge can own — all in under 60 seconds.

PAGES 2–3 | IMPLEMENTATION GUIDE

How We Build ScoutAI in 24 Hours

Architecture Overview

ScoutAI follows a clean three-layer architecture: Input → Pipeline → Output. Each layer maps directly to one of the four required tools and is independently buildable by separate team members.

LAYER 1 — INPUT	LAYER 2 — PIPELINE	LAYER 3 — OUTPUT
Bolt Frontend	Firecrawl + OpenAI	Supabase + Bolt Results UI
User types objective or pastes URLs	Scrape → Clean → Synthesise → Structure	Store brief, render card, enable share/download
~4 hours to build	~8 hours to build	~4 hours to build

Phase 1 — Setup & API Keys (Hours 0–2)

All eight team members must complete this phase before any coding starts. This is the most important step — a missing API key at hour 20 kills the demo.

- **Firecrawl:** Create Firecrawl account at firecrawl.dev → get API key → test /scrape endpoint with a sample URL using Postman or curl.
- **OpenAI:** Create OpenAI account → generate API key → confirm access to gpt-4o model. Test with a simple completion call.
- **Supabase:** Create Supabase project → note the Project URL and anon key → create table: briefs (id, created_at, objective, result_json, user_id).
- **Bolt:** Create Bolt project (bolt.new) → connect to Supabase using the Supabase integration. This is the starting point for all frontend work.

Phase 2 — Core Pipeline (Hours 2–10)

This is the engine of ScoutAI. Developers build the backend logic in three sequential steps.

Step 2A — Firecrawl Scraping Module

Build a single async function: `scrapeCompetitors(objective)`. It does two things:

- If the user provides URLs → call Firecrawl /scrape for each URL. Firecrawl returns clean markdown content, headings, and links for each page.
- If the user provides only an objective → use OpenAI first to generate a list of likely competitor domain names (e.g. 'name the top 5 SaaS tools competing in project management'). Then pass those domains to Firecrawl /scrape.

Key Firecrawl parameters to set: `formats: ['markdown'], onlyMainContent: true, timeout: 15000`. This strips boilerplate and returns clean, LLM-ready content per page.

Step 2B — OpenAI Synthesis Module

Build a single async function: `generateBrief(objective, scrapedContent)`. This calls GPT-4o with a structured system prompt. The prompt instructs the model to return a JSON object with exactly these fields:

- `summary` — 2-sentence overview of the competitive landscape
- `players` — array of objects: `{ name, positioning, key_message, pricing_signal }`
- `messaging_themes` — array of themes ALL competitors use (saturation signals)
- `gap` — the single most important thing NO competitor is saying (the white space)
- `recommended_angle` — one sentence: how the user should position themselves

Instruct GPT-4o to respond ONLY in JSON, no markdown fences, no preamble. Parse the response with `JSON.parse()` wrapped in `try/catch`.

Step 2C — Supabase Storage Module

Build a single async function: `saveBrief(userId, objective, briefJson)`. Inserts one row into the `briefs` table. On the read side, build `getBriefs(userId)` that returns all past briefs for the history panel. Use Supabase's JavaScript client — the setup is two lines once the project credentials are in place.

Phase 3 — Frontend in Bolt (Hours 10–18)

Designers and frontend developers own this phase. There are four distinct UI components to build inside Bolt.

Component 1 — Input Panel

A clean, centred input with a large text area labelled 'What do you want to know about your competitors?' Below it: a small optional field for 'Paste specific URLs (optional)'. One prominent call-to-action button: 'Generate Brief'. Include a short animated loading state with the text: 'ScoutAI is researching...' — this is a crucial demo moment.

Component 2 — Brief Results Card

This is the primary wow moment. The brief renders as a structured card with five clearly labelled sections matching the JSON fields: Landscape Summary, Who Is Playing, Dominant Messaging Themes, The Gap (highlighted in gold / amber), and Your Recommended Angle. The Gap section must be visually prominent — use a coloured callout box so judges immediately see the value.

Component 3 — History Panel

A left sidebar or bottom drawer showing past briefs pulled from Supabase. Each entry shows the objective text and timestamp. Clicking one reloads that brief. This demonstrates persistent storage and real product thinking — something most hackathon demos skip.

Component 4 — Export / Share Button

A single 'Copy Brief' or 'Download as PDF' button on the results card. Even if the PDF export is basic, this signals a production-ready mindset to judges. Use browser print-to-PDF as a fallback if a library is not ready in time.

Phase 4 — Integration & Demo Prep (Hours 18–24)

This phase is owned by the Team Lead plus two members. The goal is a seamless, rehearsed demo — not a finished product.

- **Integration:** Wire all three modules together: Input → scrapeCompetitors → generateBrief → saveBrief → render results card.
- **Demo hardening:** Prepare three pre-loaded demo scenarios with reliable outputs. Do not rely on live scraping during the demo — have fallback cached results ready if the network is slow.
- **Pitch script:** The opening line: 'Today, 88.6% of companies have no dedicated competitive intelligence team. ScoutAI gives every business analyst a CI team in their browser.' Then type the demo query live.
- **Presentation slides:** Non-tech members write this: What ScoutAI does, the gap it fills, the tech stack (at a high level), and a call to action. Keep it to 3 minutes.

Creativity — Where It Shows

The judges are venture and business AI leaders. They will judge ScoutAI on three creative signals:

- **Creative Input Model:** The input is natural language, not a URL form. This is the product insight — users think in objectives, not links. No existing scraping tool makes this leap.
- **The Gap Feature:** Every other scraping tool tells you what competitors say. ScoutAI tells you what none of them are saying. This is the inversion that makes the demo land.
- **Narrative Output:** The brief is designed to be read, not scanned. It has a narrative arc: landscape → players → themes → gap → your move. This transforms raw scraping into strategic counsel.

PAGE 4 | TEAM ROLES & RESPONSIBILITIES

8 People, 24 Hours — Who Does What

Every role below maps to a specific deliverable. No one is blocked waiting for someone else — each person can start their lane immediately after Phase 1 is complete.

#	Role	Member Type	Phase	Deliverable
1	Team Lead	Developer	All phases	Integration, coordination, demo hardening, final wiring
2	Backend Dev 1	Developer	Phase 2A	Firecrawl scraping module — <code>scrapeCompetitors()</code> function
3	Backend Dev 2	Developer	Phase 2B	OpenAI synthesis module — <code>generateBrief()</code> function + prompt engineering
4	Backend Dev 3	Developer	Phase 2C + 4	Supabase schema, <code>saveBrief()</code> , <code>getBriefs()</code> — storage layer
5	Frontend Dev	Developer	Phase 3	Bolt UI — Input Panel + Results Card components
6	Designer	Designer	Phase 3	Visual design of brief card, loading states, colour system, export styling
7	Content / QA	Non-tech	Phase 3 + 4	Write demo scenarios, test all 3 use cases, write pitch script, QA brief outputs
8	Presenter	Non-tech	Phase 4	Presentation slides, demo narrative, judge Q&A preparation

Non-technical members are not passengers. The Content/QA member is responsible for the demo scenarios that make the product look polished. The Presenter owns the narrative that makes judges remember ScoutAI above all other teams. These two roles directly determine whether you win.

Critical Path — What Cannot Be Late

- **Hour 2 deadline:** API keys for ALL four services must be in a shared `.env` file by Hour 2. Block everything else until this is done.
- **Hour 10 deadline:** The `generateBrief()` function with working JSON output. The frontend cannot render a brief without this.

- **Hour 18 deadline:** End-to-end flow working: type objective → get brief on screen. Rough UI is fine. Working pipeline is mandatory.
- **Hour 22 deadline:** Three polished demo scenarios with cached fallback results. Rehearse the 3-minute demo twice.